

Construisez Votre Propre Réseau IoT Longue Portée avec LoRa et Arduino

Un guide pratique inspiré du livre "Beginning LoRa Radio Networks with Arduino" de Pradeeka Seneviratne.



LoRa : La promesse d'une connectivité sans limites pour l'IoT.



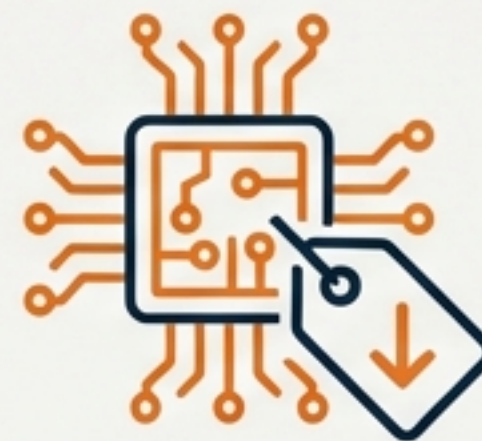
Longue Portée

Une seule passerelle peut couvrir des villes entières ou des centaines de kilomètres carrés.



Faible Consommation

Les nœuds peuvent fonctionner sur batterie pendant près de dix ans.



Coût Réduit

Infrastructure minimale, nœuds d'extrémité à faible coût et logiciels open source.

Cas d'Usage Inspirants



Suivi de bétail



Agriculture de précision



Gestion intelligente des déchets



Détection d'incendie

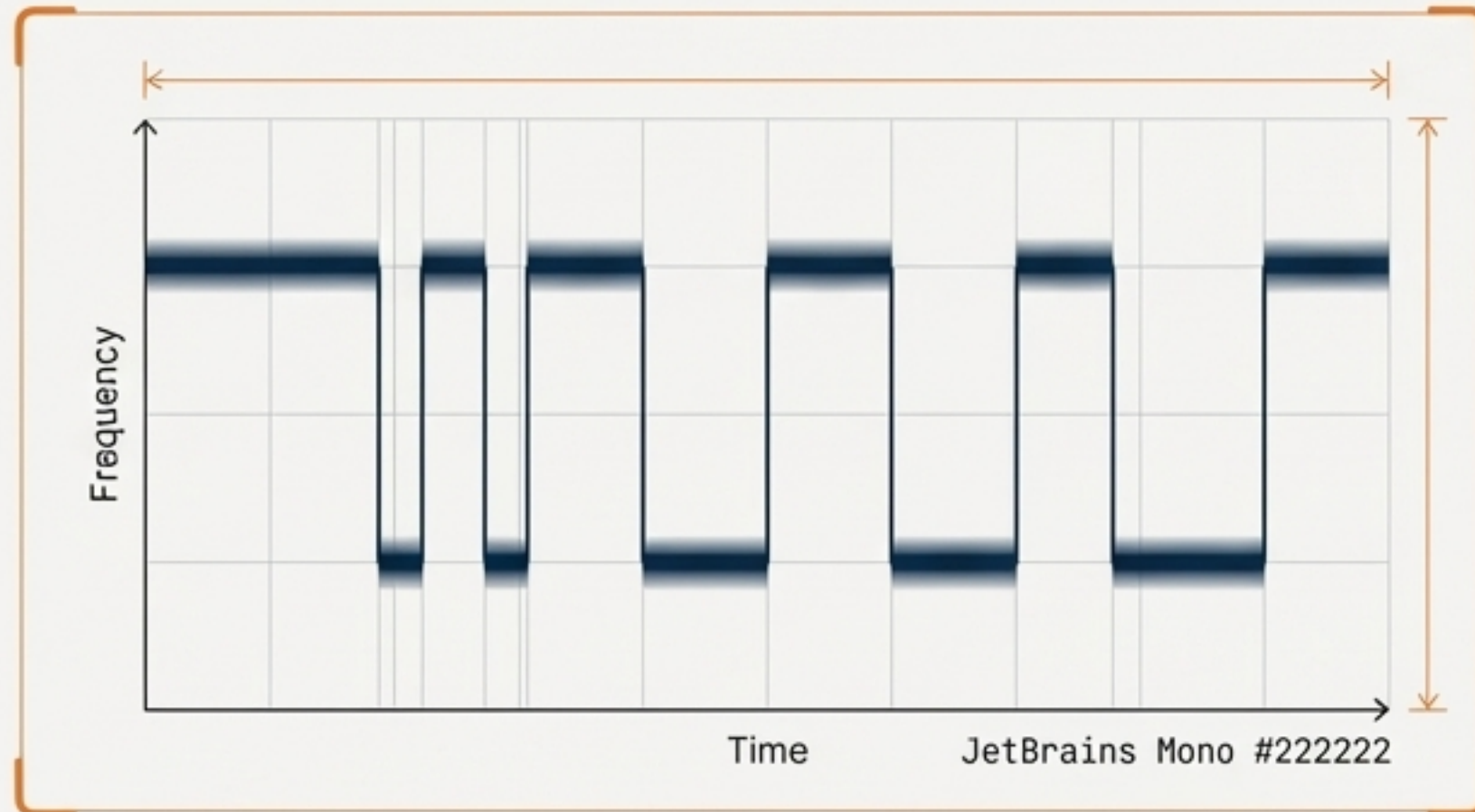


Suivi de véhicules

Le secret de la longue portée : La modulation par étalement de spectre à porteuse chirpée (CSS).

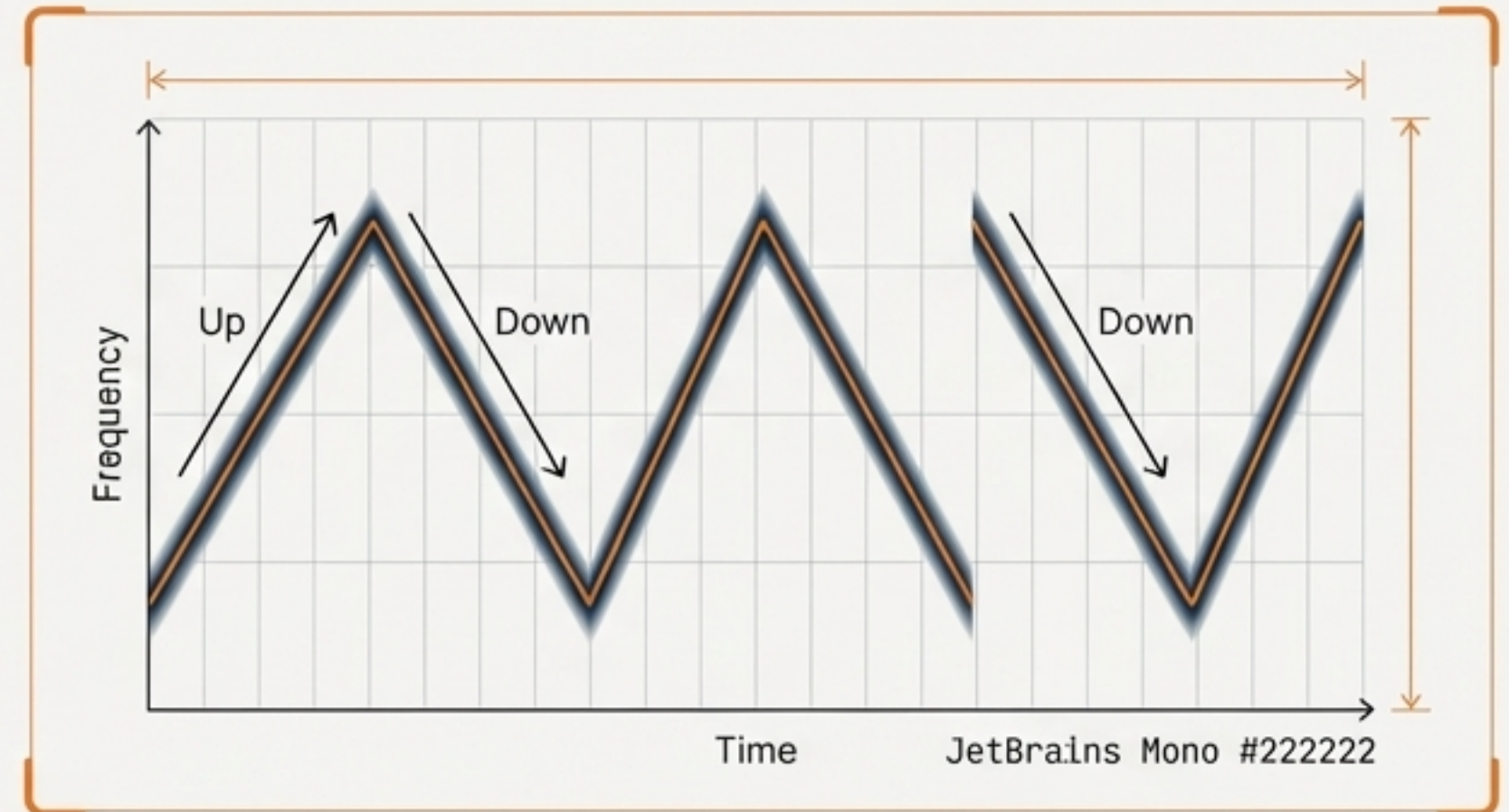
LoRa est une modulation brevetée par Semtech, robuste et efficace, basée sur le CSS. Son efficacité dépend du **Facteur d'Étalement (Spreading Factor - SF)**, qui est le nombre de 'chips' par bit, variant de 7 (débit élevé) à 12 (portée maximale).

Inter Bold modulation



FSK : Sauts brusques entre deux fréquences.
Inter Regular #222222

LoRa (CSS) modulation

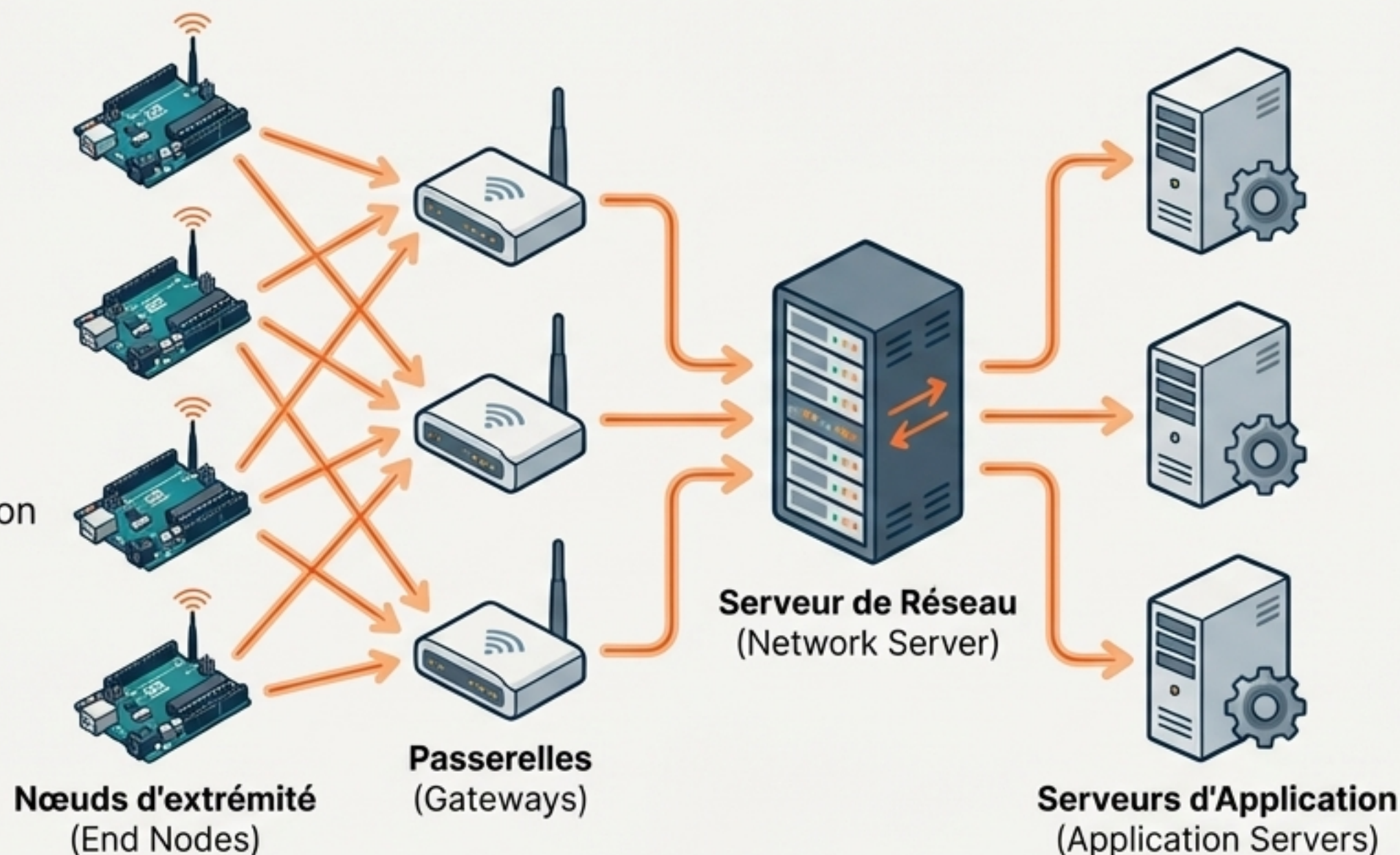


LoRa (CSS) : Balayage continu entre les fréquences pour une robustesse au bruit exceptionnelle.
Inter Regular #222222

LoRa est la technologie physique, LoRaWAN est l'architecture réseau.

LoRa (La Couche Physique)

Décrit comment le signal est envoyé.
Responsable de la liaison de communication longue portée.

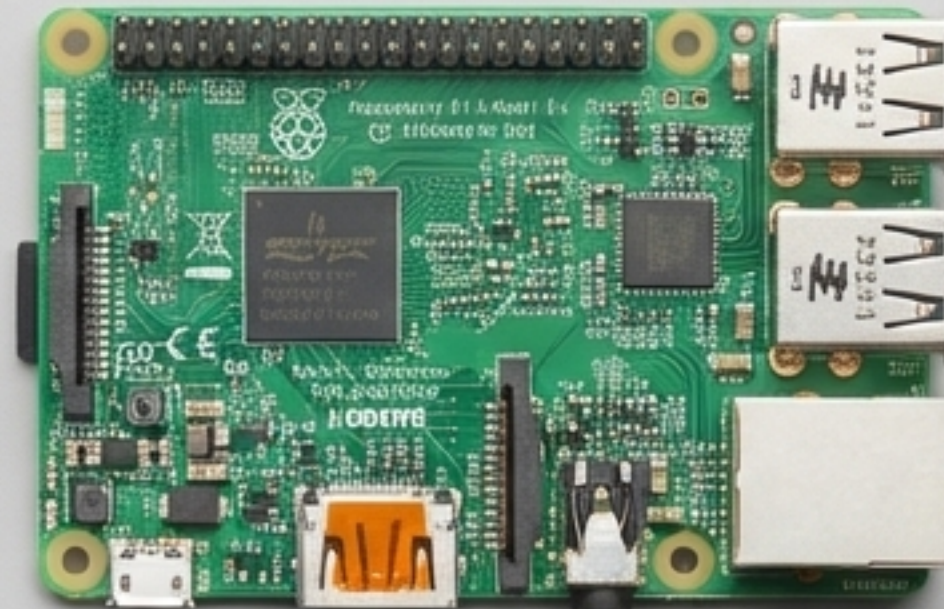
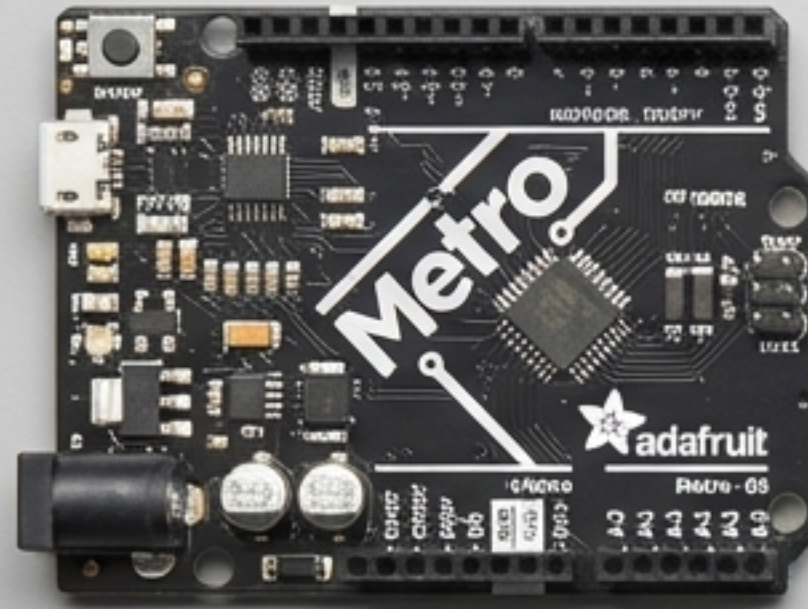


LoRaWAN (Le Protocole de Communication)

Gère l'architecture réseau et le protocole.

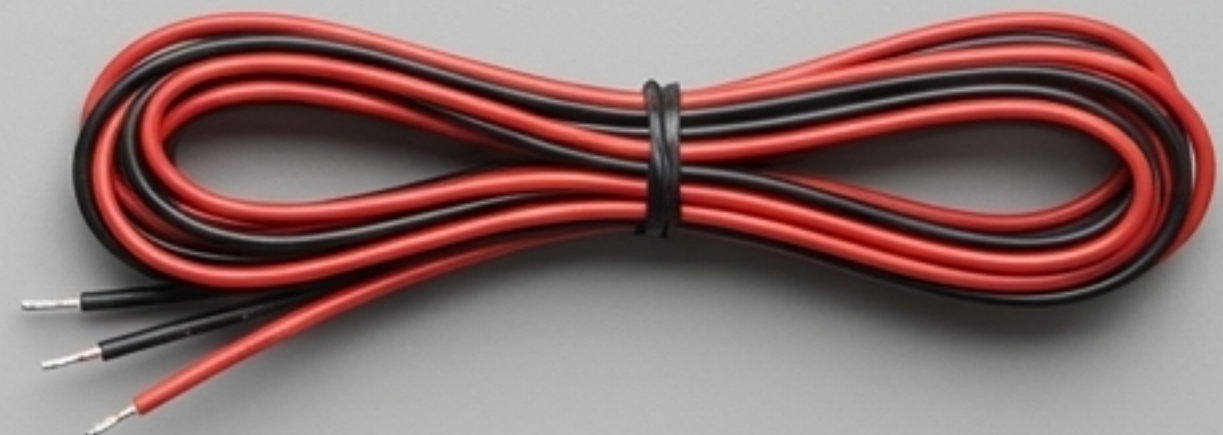
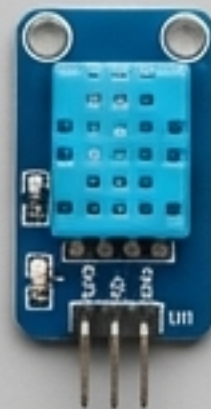
Impacte :

- Durée de vie de la batterie
- Capacité du réseau
- Qualité de service (QoS)
- Sécurité

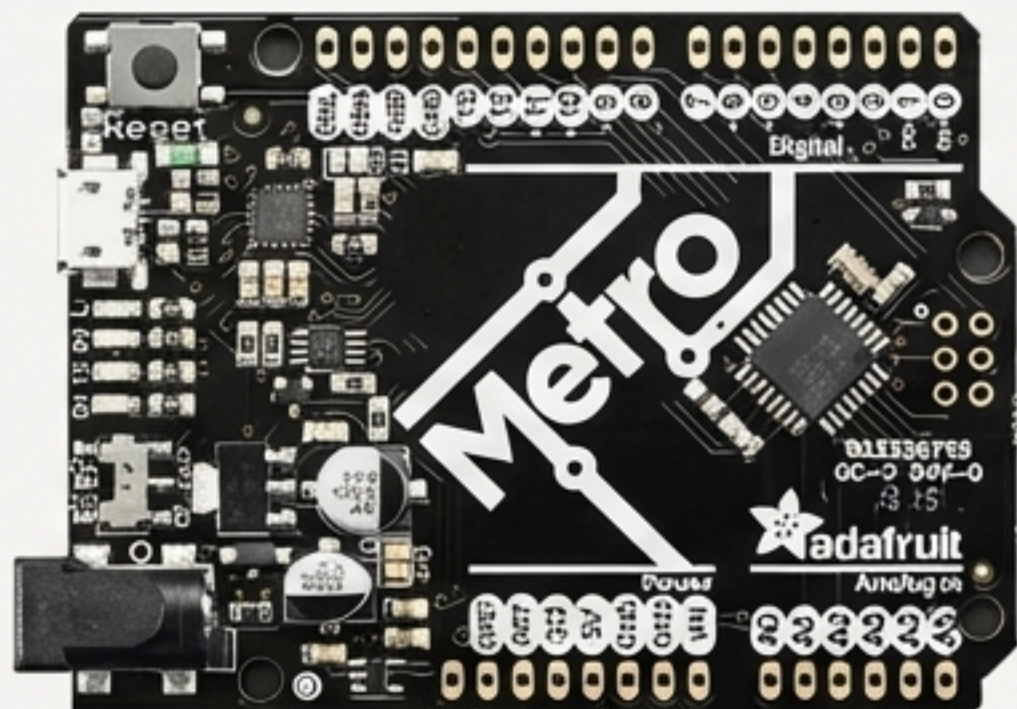


L'Atelier : Matériel et Logiciel

Rassemblons les outils nécessaires
pour commencer à construire.



Les Nœuds d'Extrémité : Les sens de votre réseau.



Microcontrôleur

Modèles: Arduino Uno ou Adafruit Metro

Le cerveau du nœud, basé sur l'ATmega328P.

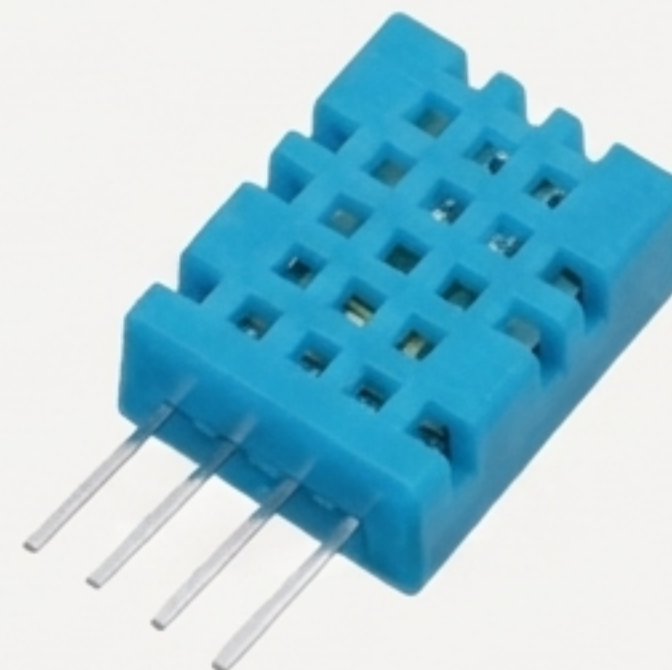


Émetteur-Récepteur Radio

Modèles: Breakouts Adafruit RFM9x

Le cœur de la communication LoRa.

- **RFM95W (SX1276)** : Pour les bandes ISM 868 MHz (Europe) et 915 MHz (Amérique).
- **RFM98W (SX1278)** : Pour la bande ISM 433 MHz.

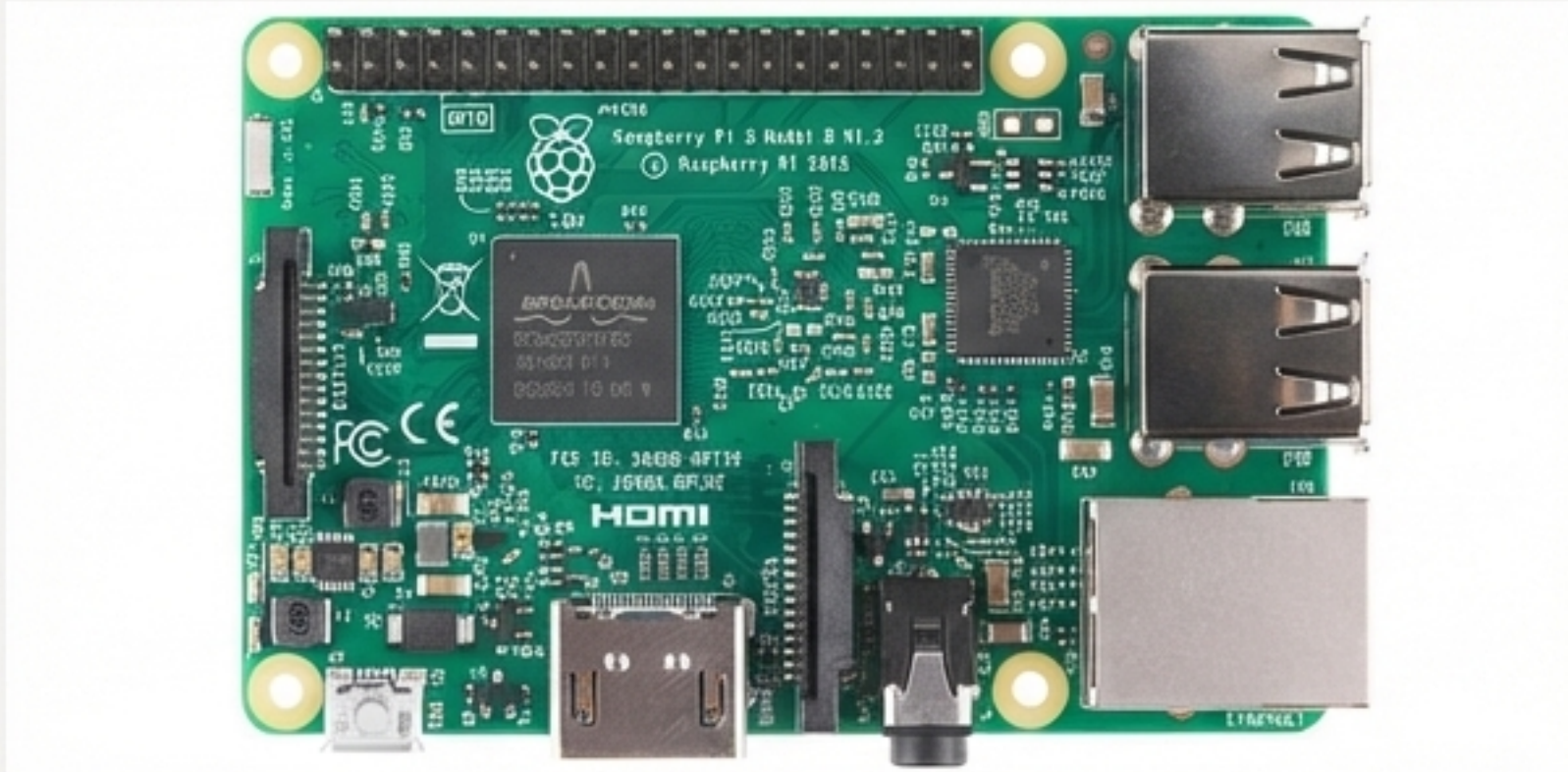


Capteurs

Modèles: Exemples pour nos projets.

- **DHT11**: Capteur de température et d'humidité de base.
- **Module GPS**: Pour le suivi de la géolocalisation.

La Passerelle : Le pont entre vos objets et Internet.



Micro-ordinateur

Raspberry Pi 3. Offre la puissance de traitement nécessaire pour gérer le trafic réseau et exécuter le logiciel 'packet forwarder'.

Émetteur-Récepteur Radio

Le même module RFM9x utilisé pour les nœuds d'extrémité peut servir à construire une passerelle monocanal.



Connexion Internet

La passerelle utilise le Wi-Fi ou l'Ethernet pour se connecter à Internet et transférer les paquets LoRa vers un serveur de réseau.

Note importante : Les passerelles monocanal sont idéales pour l'apprentissage et les tests, mais ne sont pas entièrement conformes à la spécification LoRaWAN, qui requiert des concentrateurs multicanaux (basés sur des puces comme le SX1301).

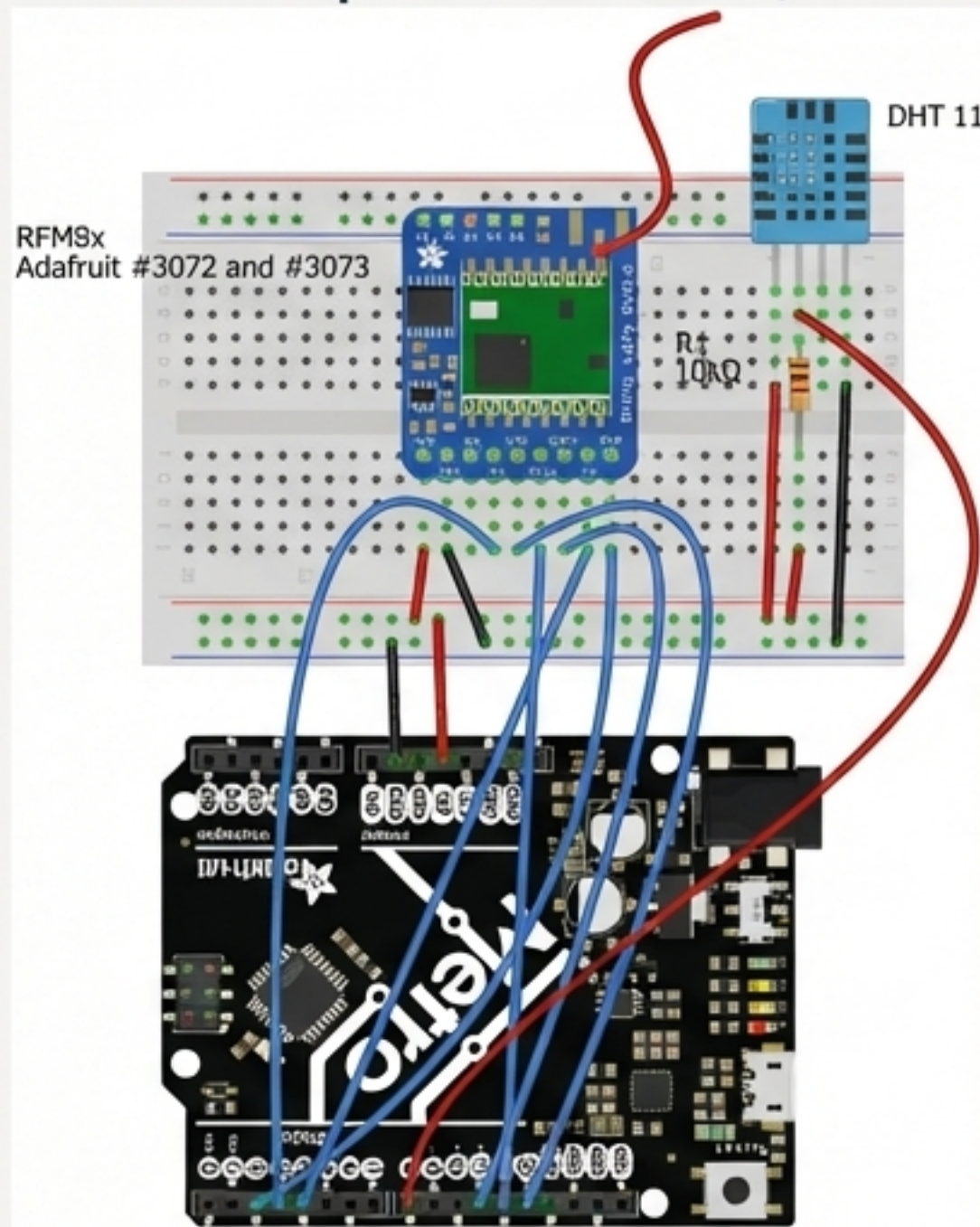
Première Création : La Communication Directe

Établissons une liaison point à point simple entre deux nœuds LoRa.



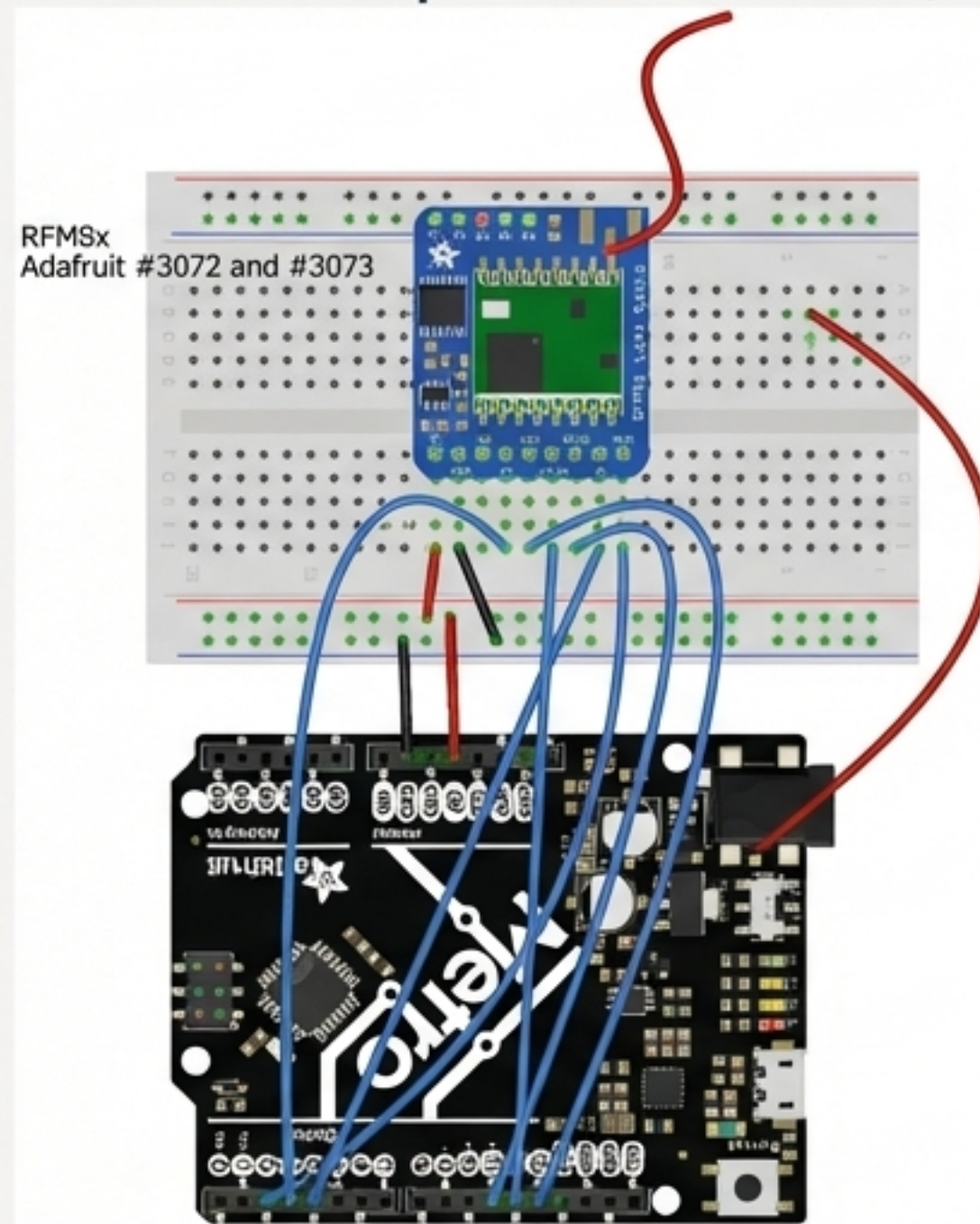
L'assemblage : Schémas de câblage pour le nœud capteur et le nœud récepteur.

Nœud Capteur (Client)



Arduino	RFM9x	DHT11
VIN	VIN	VIN
GND	GND	GND
2	RST	
3	G0	
4	CS	
8		DATA
11	MOSI	
12	MISO	
13	SCK	

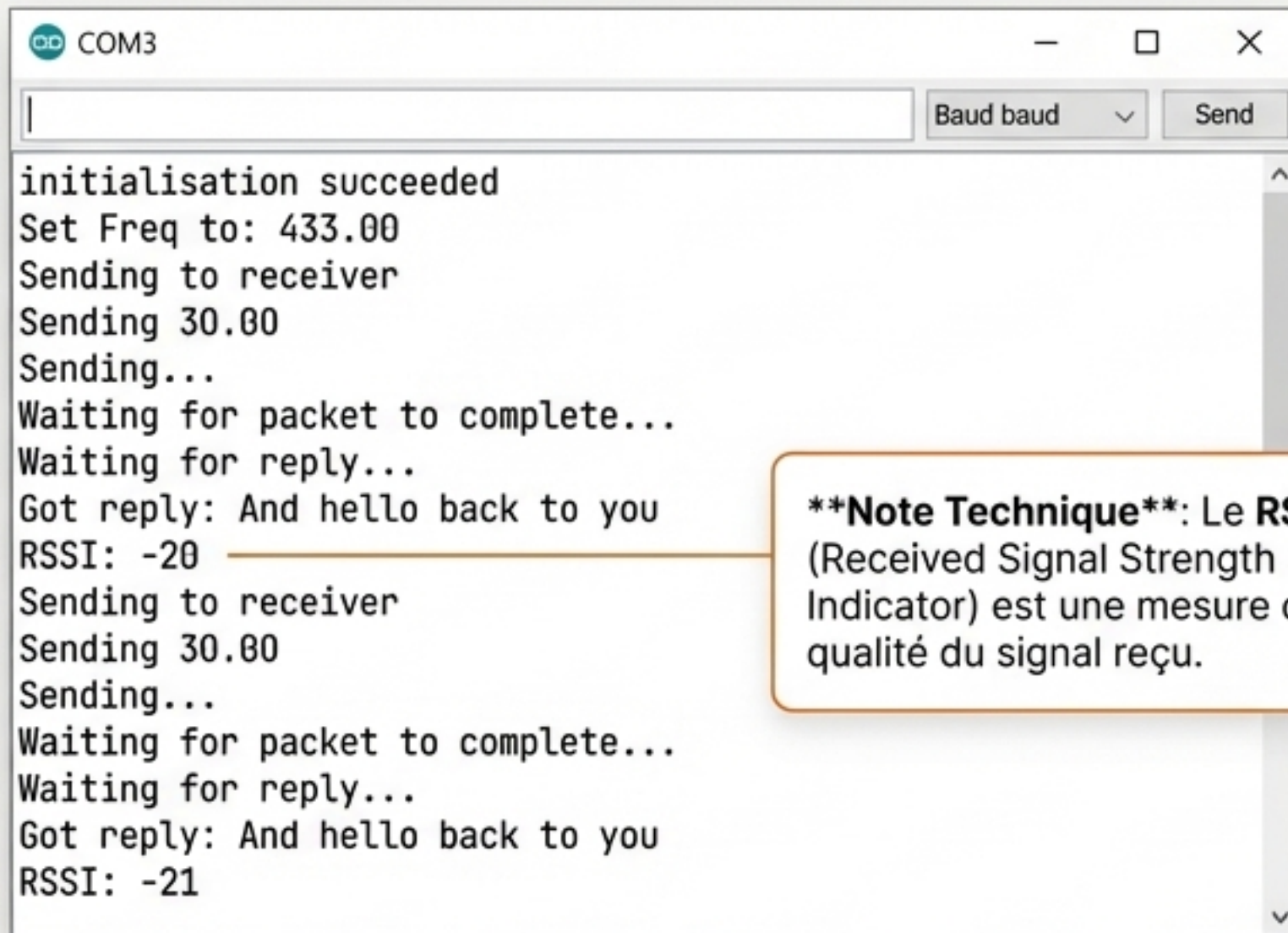
Nœud Récepteur (Serveur)



Arduino	RFM9x
VIN	VIN
GND	GND
2	RST
3	G0
4	CS
11	MOSI
12	MISO
13	SCK

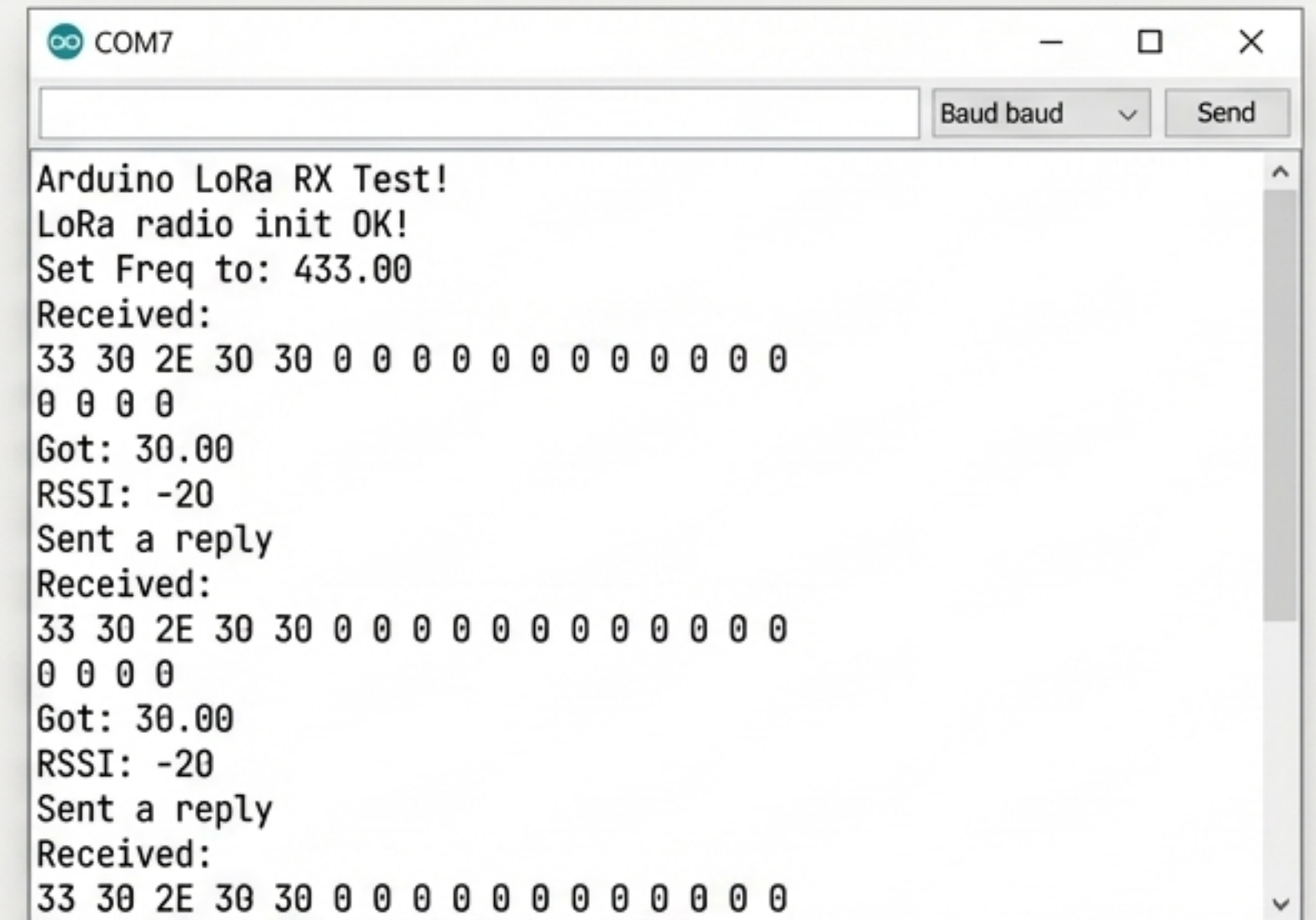
Le résultat : Une conversation bidirectionnelle réussie.

Le nœud capteur envoie la température (ex: 30.00°C) et attend une réponse. Le nœud récepteur reçoit les données, les affiche, et renvoie un message d'accusé de réception.



```
COM3
initialisation succeeded
Set Freq to: 433.00
Sending to receiver
Sending 30.00
Sending...
Waiting for packet to complete...
Waiting for reply...
Got reply: And hello back to you
RSSI: -20
Sending to receiver
Sending 30.00
Sending...
Waiting for packet to complete...
Waiting for reply...
Got reply: And hello back to you
RSSI: -21
```

****Note Technique****: Le **RSSI** (Received Signal Strength Indicator) est une mesure de la qualité du signal reçu.



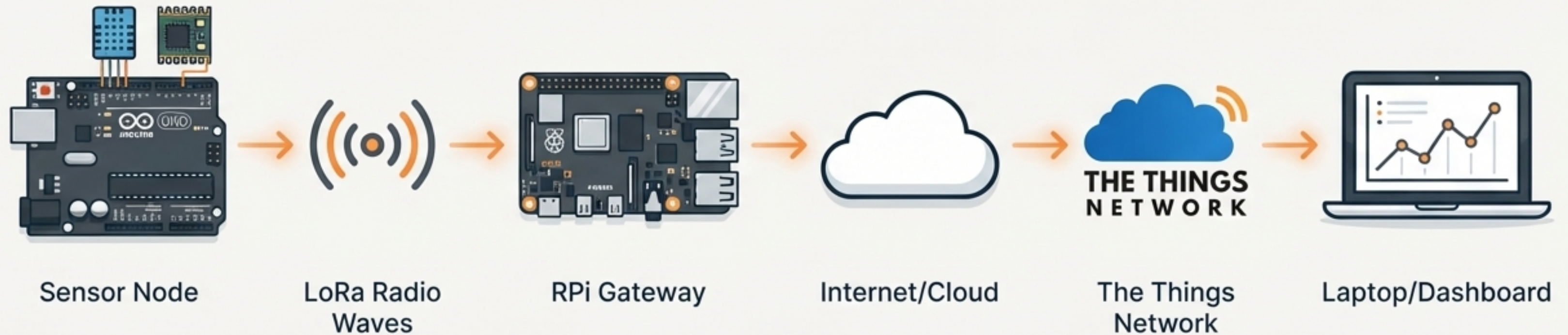
```
COM7
Arduino LoRa RX Test!
LoRa radio init OK!
Set Freq to: 433.00
Received:
33 30 2E 30 30 0 0 0 0 0 0 0 0 0 0
0 0 0 0
Got: 30.00
RSSI: -20
Sent a reply
Received:
33 30 2E 30 30 0 0 0 0 0 0 0 0 0 0
0 0 0 0
Got: 30.00
RSSI: -20
Sent a reply
Received:
33 30 2E 30 30 0 0 0 0 0 0 0 0 0 0
```

Visuel Gauche (Moniteur Série du Nœud Capteur)

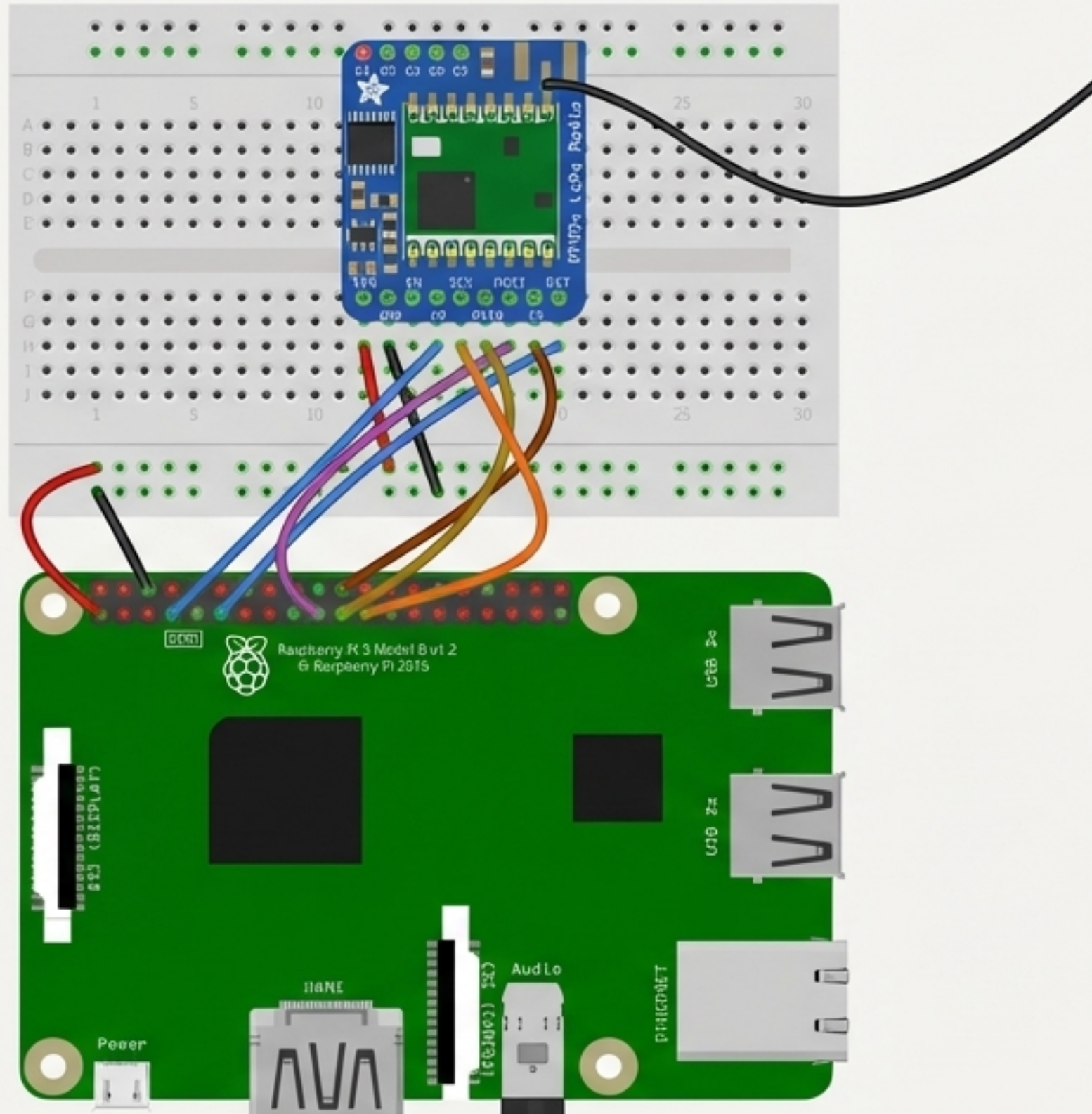
Visuel Droit (Moniteur Série du Nœud Récepteur)

Le Chef-d'œuvre : La Passerelle IoT Connectée

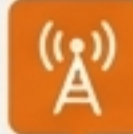
Connectons notre capteur à Internet via une passerelle Raspberry Pi et The Things Network.



Assemblage de la passerelle Raspberry Pi monocanal.



Raspberry Pi Pin	RFM9x Pin
1 (3.3V)	`VIN`
6 (GND)	`GND`
7 (GPIO4)	`G0`
11 (GPIO17)	`RST`
19 (MOSI)	`MOSI`
21 (MISO)	`MISO`
22 (GPIO25)	`CS`
23 (SCLK)	`SCK`



N'oubliez pas de souder une antenne filaire quart d'onde à votre module RFM9x ! (ex: 16.5 cm pour 433 MHz, 8.2 cm pour 868 MHz).

Configuration logicielle : Donner vie à votre passerelle.



Préparer la carte SD

Flasher l'OS Raspbian Stretch LITE avec Etcher.
Créer les fichiers `wpa_supplicant.conf` et `ssh` à la racine.



Premier Démarrage & Accès

Se connecter en SSH via PuTTY.
Mettre à jour le système (`sudo apt-get update`).



Activer les Interfaces

Utiliser `sudo raspi-config` pour activer l'interface SPI, essentielle pour la communication LoRa.



Installer les Dépendances

Installer `git` et la bibliothèque `wiringpi`.



Installer le Packet Forwarder

Cloner le dépôt `single_chan_pkt_fwd`.

⚠ Point Crucial: Éditer `main.cpp` pour définir la fréquence (`freq`) et l'adresse du serveur (`SERVER1`).

```
pi@raspberrypi: ~ - PuTTY
pi@raspberrypi:~/single_chan_pkt_fwd $ sudo /home/pi/single_chan_pkt_fw
d/single_chan_pkt_fwd
SX1276 detected, starting.
Gateway ID: b8:27:eb:ff:ff:7a:51:d5

Listening at SF7 on 433.100000 Mhz.
-----
stat update: {"stat":{"time":"2018-10-19 06:50:56 GMT","lati":0.00000,"
ong":0.00000,"alti":0,"rxnb":0,"rxok":0,"rxfw":0,"ackr":0.0,"dwnb":0,"t
nb":0,"pfrm":"Single Channel Gateway","mail":"","desc":""}}
stat update: {"stat":{"time":"2018-10-19 06:51:26 GMT","lati":0.00000,"
ong":0.00000,"alti":0,"rxnb":0,"rxok":0,"rxfw":0,"ackr":0.0,"dwnb":0,"t
nb":0,"pfrm":"Single Channel Gateway","mail":"","desc":""}}
stat update: {"stat":{"time":"2018-10-19 06:51:56 GMT","lati":0.00000,"
ong":0.00000,"alti":0,"rxnb":0,"rxok":0,"rxfw":0,"ackr":0.0,"dwnb":0,"t
nb":0,"pfrm":"Single Channel Gateway","mail":"","desc":""}}
```


Enregistrement sur The Things Network (TTN).

Processus d'Enregistrement de la Passerelle

1. Créer un compte sur thethingsnetwork.org et accéder à la Console.
2. Aller dans la section "Gateways" et cliquer sur "register gateway".
3. Remplir les informations critiques :
 - Cocher "**I'm using the legacy packet forwarder**".
 - **Gateway EUI**: Entrer l'ID généré par le script (ex: b827ebffff7a51d5).
 - **Frequency Plan**: Doit correspondre à votre configuration.
 - **Router**: Choisir le routeur le plus proche.

GATEWAY OVERVIEW

Gateway ID eui-b827ebffff7a51d5

Description My First Raspberry Pi Gateway

Owner  pradeeka7  [Transfer ownership](#)

Status ● connected

Frequency Plan Europe 868MHz

Router ttn-router-eu

Gateway Key 

Last Seen 5 seconds ago

Received Messages 

Transmitted Messages 0

Le cycle complet : Du capteur au cloud en temps réel.

1. Créer une Application et un Appareil sur TTN

- Dans la console TTN, créer une 'Application', puis enregistrer un 'Device'.
- Configurer la méthode d'activation sur **ABP (Activation By Personalization)**. Copier les clés générées : `Device Network Session Key`, et `App Session Key`. Ces clés seront insérées dans le code Arduino du nœud capteur.

2. Voir les Données Arriver

Le nœud envoie les données chiffrées. La passerelle les transmet à TTN. L'onglet 'Data' de votre application affiche les paquets entrants et leur contenu décodé.

The image shows a workflow for sending data from an Arduino sensor to the TTN cloud. On the left, the JetBrains Mono code editor displays Arduino code for an ABP network. The code defines a device address of 0x26011 and keys. An orange arrow points from the device address in the code to the 'PAYLOAD FORMATS' section in the center. This section shows a 'Custom' payload format with a JavaScript decoder function. Another orange arrow points from the decoder's output to the 'TTN Console' on the right. The console's 'Data' tab shows an uplink message at 15:45:12 with a frame of 33 and port 1. The payload is shown in hexadecimal (32 39 2E 30 2C 38 33 2E 30) and then decoded into a JSON object: { message: "29.0,83.0" }. A 'NotebookLM' watermark is visible in the bottom right corner.

```
#include <lmic.h>

static const u1_t NWKSKEY[16] = { 0xAD, ... }
static const u1_t APPSKEY[16] = { 0xBE, ... }
static const u4_t DEVADDR = 0x26011...;
static const u4_t DEVADDR = 0x26011...;
```

PAYLOAD FORMATS

Payload Format
The payload format sent by your devices

Custom

decoder converter validator encoder

```
1 function Decoder(bytes, port) {
2   // Decode an uplink message from a buffer
3   return {
4     message: String.fromCharCode.apply(null, bytes)
5   };
6 }
```

TTN Console

Overview Data Feeding Connect

15:45:12 | Uplink | Frame: 33 | Port: 1 | ...

payload

32 39 2E 30 2C 38 33 2E 30

fields ⓘ

```
{
  message: "29.0,83.0"
}
```

NotebookLM